

D'accord, vas-y, je t'écoute.

uh, Tu sais, ce qu'on a parlé ce matin par rapport à notre modèle. Et du coup, uh au niveau de réflexion, euh au niveau de du brain et puis etc. euh Là, je suis en train de de revoir un petit peu dans ma tête, le système d'OCR. Et tu sais que on avait dit euh, je sais pas si tu te rappelles mais il y avait une sorte de modèle qui interrogeait le RAG, tu avais de l'ingestion euh qui faisait la découpe de shrunk. uh du small myotique et tout ça

Oui. Oui.

Et du coup euh la-la question c'était euh ok c'est super mais euh est-ce que j'ai bien pensé à mettre à créer une base de données par projet pour que en gros euh ça soit vraiment une une tour de contrôle qui permette de dire bah écoute je vais ingérer ça je vais le tester. Donc du coup je suis isolé parce que je suis pas en train de polluer toutes les bases de données ou toutes les collections. Du coup euh à chaque fois que je décide je crée une base de données je fais un projet d'injection, j'interroge l'injection si elle est correcte

Hmm. Je vois ce que tu veux dire, et oui, très bonne direction. En fait, ce que je ferais à ta place, c'est carrément isoler un dossier de travail par projet. Un workspace autonome par projet, avec sa config, ses documents, ses chunks et ses embeddings, ses logs, etcétera. Et comme ça, ta base de données ne sert qu'à ce projet. Tu peux versionner, comparer plusieurs approches, et faire tes tests en parallèle. Et si tu veux ensuite fusionner ce qui a marché, tu peux toujours pousser les projets validés vers une base commune de connaissances que tu sais qui est fiable. Franchement, c'est une bonne approche, structurée et propre. Je te donnerai 5 sur 5 pour

la cohérence et la clarté, surtout si ton but est de faire des tests sans contaminer le reste.

ok. Donc du coup, uh arrêtons de faire de du proxy frontal, c'est rigolo mais on va aller à proposer. Qu'est-ce que je veux dire? uh pour mon projet de, on avait dit quoi? que il fallait que j'ai une sorte de PRD uh et de documentation pour dire pour te rappeler ce soir uh ça va juste un mot rappeler. C'était quoi déjà

Je pense

Complètement d'accord. Le secret, ce n'est pas d'empiler des briques, c'est de les découpler. Chaque couche ne connaît que son contrat d'entrée et de sortie. Pas ses voisins. Tu peux même définir une règle simple : tout est un plugin. Activable ou non. Et piloté par config.

Document, schéma, template, policy checker, brain, paradigms, methodes, consensus, validation, knowledge state manager. Chacun renvoie un objet standardisé. Et si tu ajoutes un principe d'« architecture en couches facultatives », tu peux en enlever une sans faire s'écrouler le reste. Et je pense que les deux docs les plus importants de ton projet ne seront pas le PRD ni le pseudo-code, mais le manifeste d'architecture et le registre. Si ces deux là sont solides, tu pourras faire évoluer tout le reste sans tout casser.

Ok vas y

Minute, je lis. Parfait. On peut formaliser ça proprement. Voici un manifeste d'architecture V0. Règle 1 : tout est module, rien en dur. Règle 2, tout communique par contrat avec des entrées-sorties strictes. Règle 3, un projet égale un workspace isolé. Règle 4, le Registry sait tout, le Brain choisit, mais ne contient rien. Règle 5, Policy Checker valide toute action critique. Règle 6, le State Manager limite les recalculs. Et la règle7 : l'optimiseur de fond

consolide sans perturber. Et franchement... Si tu bâtis autour de ces principes, tu n'auras pas un château de cartes, tu auras une charpente.

Tu veux conserver une synthèse très détaillée de cette conversation, qui contient deux séquences distinctes :

La réflexion sur l'architecture de ton système OCR-RAG et de ta « tour de contrôle » IA.

Le compte rendu de ta visite médicale professionnelle.

1. Architecture du système OCR-RAG

Contexte de la réflexion

tu repenses au système discuté le matin autour :

du modèle de raisonnement ;

du Brain ;

de l'ingestion documentaire ;

de l'OCR ;

de la découpe des documents en chunks ;

de l'enrichissement maïeutique des chunks ;

du stockage dans un RAG ;

de l'interrogation et de la validation des connaissances ingérées.

Ta préoccupation principale est de vérifier que l'architecture ne mélange pas toutes les données dans une seule base générale.

Idée centrale : une base isolée par projet

Tu proposes qu'à chaque nouveau projet d'ingestion, le système crée un environnement indépendant.

Un projet d'ingestion disposerait donc de :

sa propre base de données ou collection ;

ses documents sources ;

ses chunks ;

ses embeddings ;

ses paramètres d'ingestion ;

ses traitements d'enrichissement ;

ses résultats de tests ;

ses journaux d'exécution ;

ses métriques ;

ses versions ;

ses validations.

L'objectif est d'éviter qu'une expérimentation ou qu'une ingestion incorrecte :

pollue les connaissances d'un autre projet ;

contamine une base globale ;

rende les résultats impossibles à auditer ;

crée des conflits entre plusieurs corpus ;

mélange des versions différentes d'une même connaissance.

Le principe formulé est donc :

Un projet = un workspace isolé = une base ou collection dédiée.

Cycle de vie d'un projet d'ingestion

Le fonctionnement imaginé est le suivant :

Création d'un nouveau projet.

Création automatique de son workspace.

Configuration de l'ingestion.

Ajout des documents.

OCR si nécessaire.

Nettoyage et normalisation du contenu.

Découpage en chunks.

Enrichissement des chunks :

métadonnées ;

tags ;

concepts ;

contexte ;

analyse maïeutique ;

relations ;

éventuellement pseudo-code, logique ou philosophie.

Création des embeddings.

Stockage dans la base isolée du projet.

Interrogation du RAG.

Tests de qualité.

Vérification de la pertinence des réponses.

Contrôle des sources et des citations.

Validation ou rejet de l'ingestion.

Comparaison avec d'autres configurations.

Promotion éventuelle du projet vers une base commune validée.

Ainsi, une ingestion n'est jamais considérée comme fiable uniquement parce qu'elle a terminé sans erreur technique. Elle doit être interrogée, testée et validée.

Comparaison de plusieurs méthodes

L'isolation par projet permet également de créer plusieurs variantes du même corpus.

Par exemple :

projet A avec des chunks courts ;

projet B avec des chunks longs ;

projet C avec recouvrement important ;

projet D avec enrichissement maïeutique ;

projet E sans enrichissement ;

projet F avec un autre modèle d'embedding ;

projet G avec un autre système de tags.

Ces projets peuvent fonctionner en parallèle sans se contaminer.

Tu peux alors comparer :

la pertinence des réponses ;

le taux de récupération des bonnes sources ;

le nombre d'hallucinations ;

le coût en tokens ;

le temps d'ingestion ;

la taille de la base ;

la précision des citations ;

la qualité des relations entre chunks ;

la stabilité des réponses ;

la capacité à retrouver une information très précise.

Une fois la meilleure configuration identifiée, elle peut devenir une configuration officielle ou être transférée vers une base commune.

2. Principe de promotion vers une base fiable

La base globale ne doit pas recevoir automatiquement toutes les données traitées.

Elle doit recevoir uniquement :

les projets validés ;

les documents contrôlés ;

les chunks conformes ;

les connaissances dont la provenance est connue ;

les résultats ayant passé les tests ;

les versions approuvées.

On distingue donc plusieurs états possibles :

brouillon ;

en ingestion ;

en cours de traitement ;

prêt pour les tests ;

en validation ;

rejeté ;

validé ;

archivé ;

promu vers la base commune.

Cette séparation transforme la base globale en base de connaissances certifiée, plutôt qu'en simple stockage de tout ce qui a été ingéré.

3. Architecture modulaire : éviter le château de cartes

La discussion insiste ensuite sur un risque important : construire un système composé de nombreuses couches tellement dépendantes les unes des autres que la modification d'un élément provoque l'effondrement de l'ensemble.

La réponse proposée est le découplage.

Chaque composant doit connaître uniquement :

son contrat d'entrée ;

son contrat de sortie ;

ses responsabilités ;

ses permissions ;

ses erreurs possibles.

Un module ne doit pas dépendre directement du fonctionnement interne de ses voisins.

Par exemple, le module de chunking ne doit pas avoir

besoin de connaître :

le fournisseur du modèle d'embedding ;

la structure interne du Brain ;

le fonctionnement du moteur de recherche ;

la logique du validateur final.

Il reçoit un objet standardisé et renvoie un objet standardisé.

Principe : tout est un plugin

Tous les composants doivent être :

indépendants ;

configurables ;

activables ;

désactivables ;

remplaçables ;

versionnables ;

testables séparément.

Les éléments évoqués comme plugins ou services sont notamment :

ingestion documentaire ;

OCR ;

normalisation ;

chunking ;

enrichissement ;

schémas ;

templates ;

embeddings ;

recherche vectorielle ;

recherche lexicale ;

reranking ;

Policy Checker ;

Brain ;

méthodes de raisonnement ;

paradigmes ;

consensus ;

validation ;

gestion de l'état des connaissances ;

journalisation ;

optimisation de fond.

L'objectif est qu'une couche puisse être retirée ou remplacée sans faire tomber l'ensemble.

4. Les objets standardisés

Chaque module doit renvoyer un objet conforme à un schéma commun.

Cet objet pourrait contenir :

```
{  
  "project_id": "project_x",  
  "module_id": "chunk_enricher",  
  "module_version": "1.0.0",
```

```
"input_type": "normalized_document",
"output_type": "enriched_chunks",
"status": "success",
"data": {},
"metadata": {},
"sources": [],
"warnings": [],
"errors": [],
"metrics": {
  "duration_ms": 0,
  "token_cost": 0,
  "confidence": 0
},
"trace_id": "trace_x",
"created_at": "timestamp"
}
```

Cette standardisation apporte :

de la traçabilité ;

une compatibilité entre modules ;

des tests plus simples ;

une meilleure gestion des erreurs ;

la possibilité de rejouer une étape ;

la comparaison de plusieurs versions ;

l'audit complet d'une réponse.

5. Les documents structurants du projet

Tu te demandais quel document ou quel mot devait te permettre de reprendre le projet le soir.

Plusieurs livrables sont évoqués :

le PRD ;

le Dev Book ;

le pseudo-code ;

le Dev Tracker ;

le manifeste d'architecture ;

le registre des composants.

La conclusion importante est que les deux documents les plus structurants ne sont pas nécessairement le PRD ou le pseudo-code.

5.1 Le manifeste d'architecture

Il décrit les règles invariantes du système.

Il répond notamment aux questions :

Quelles règles aucun module ne doit-il enfreindre ?

Comment les composants communiquent-ils ?

Comment les données sont-elles isolées ?

Qui peut prendre une décision ?

Qui valide les actions critiques ?

Comment éviter les recalculs inutiles ?

Comment faire évoluer le système sans le casser ?

5.2 Le registre ou Registry

Le Registry constitue le catalogue central des capacités disponibles.

Il connaît :

les modules installés ;

leurs versions ;

leurs contrats ;

leurs entrées ;

leurs sorties ;

leurs capacités ;

leurs dépendances ;

leurs statuts ;

leurs permissions ;

leurs coûts ;

leurs performances ;

leurs conditions d'activation.

Le Registry sait ce qui existe, mais ne décide pas nécessairement quoi utiliser.

6. Manifeste d'architecture V0

Sept règles principales ont été formulées.

Règle 1 — Tout est module, rien n'est codé en dur
Les comportements doivent être pilotés par :

configuration ;

manifeste ;

schéma ;

Registry ;

règles de politique.

Il faut éviter les conditions dispersées et les dépendances invisibles directement écrites dans le code.

Règle 2 — Communication par contrats stricts

Chaque module possède :

une entrée définie ;

une sortie définie ;

un schéma de validation ;

des codes d'erreur ;

des métadonnées ;

une version.

Deux modules ne doivent pas avoir besoin de connaître leurs mécanismes internes respectifs.

Règle 3 — Un projet égale un workspace isolé

Chaque projet dispose de :

son identité ;

sa configuration ;

son corpus ;

sa base ;

ses index ;

ses modèles éventuels ;

ses logs ;

ses tests ;

ses résultats ;

ses versions ;

ses droits d'accès.

Cette règle protège les autres projets contre les erreurs ou contaminations.

Règle 4 — Le Registry sait tout, le Brain choisit, mais ne

contient rien

Le Registry connaît les services disponibles.

Le Brain :

analyse la situation ;

identifie le besoin ;

choisit les capacités pertinentes ;

construit ou pilote le plan d'exécution ;

déclenche les modules.

En revanche, le Brain ne doit pas devenir :

une base documentaire ;

un entrepôt de chunks ;

un stockage de modèles ;

un registre de configurations ;

un composant monolithique qui fait lui-même tous les traitements.

Il orchestre mais ne remplace pas les outils spécialisés.

Règle 5 — Le Policy Checker valide les actions critiques

Le Policy Checker intervient avant les opérations à risque, notamment :

suppression ;

écrasement ;

modification de la base ;

fusion de projets ;

promotion dans la base globale ;

téléchargement externe ;

exécution de commandes ;

accès à des données sensibles ;

changement de modèle ;

appel coûteux ;

action irréversible.

Il vérifie :

les permissions ;

le périmètre du projet ;

les règles de sécurité ;

la conformité du plan ;

le coût prévu ;

les conséquences possibles ;

la nécessité d'une confirmation humaine.

Règle 6 — Le State Manager limite les recalculs

Le State Manager connaît l'état exact de chaque ressource et de chaque traitement.

Il peut déterminer :

ce qui a déjà été calculé ;

ce qui a changé ;

ce qui est encore valide ;

ce qui doit être recalculé ;

ce qui peut être réutilisé ;

ce qui doit être invalidé.

Exemple : lorsqu'une seule page change, il ne faut pas retraiter les 400 pages du document.

Le système peut recalculer uniquement :

la page modifiée ;

ses chunks ;

ses embeddings ;

ses relations ;

les index concernés ;

les synthèses qui dépendent réellement de cette page.

Règle 7 — L'optimiseur de fond consolide sans perturber
Un service secondaire peut améliorer progressivement les données sans bloquer l'utilisation principale.

Il peut notamment :

détecter les doublons ;

rapprocher les chunks similaires ;

vérifier les métadonnées ;

reconstruire certains index ;

compacter la base ;

analyser les logs ;

repérer les incohérences ;

proposer des fusions ;

recalculer certains scores ;

signaler les connaissances obsolètes.

Cet optimiseur ne doit pas modifier silencieusement une connaissance validée. Les transformations importantes doivent rester :

traçables ;

versionnées ;

réversibles ;

contrôlées par les politiques.

7. Conclusion architecturale

Le message principal est que tu ne construis pas simplement un système OCR ou un chatbot RAG.

Tu conçois une plateforme d'expérimentation, de validation et de gouvernance des connaissances, capable de :

créer un environnement isolé pour chaque projet ;

tester plusieurs pipelines ;

comparer les résultats ;

garantir la provenance des connaissances ;

empêcher la pollution entre corpus ;

valider une ingestion avant publication ;

promouvoir uniquement les connaissances fiables ;

remplacer les composants sans reconstruire tout le système ;

fournir un historique complet des décisions.

La métaphore finale est importante :

Avec ces principes, le système ne devient pas un château de cartes, mais une charpente modulaire.

Synthèse générale

Cette conversation fait apparaître deux préoccupations parallèles :

Sur le plan technique, tu cherches à rendre ton projet OCR–RAG robuste, modulaire, auditable et capable d'évoluer sans destruction

Évaluation de la cohérence : 4,8/5

La vision technique est particulièrement cohérente : l'isolation par projet, les contrats standardisés, le Registry, le Brain, le Policy Checker et le State Manager forment une architecture solide. Le prochain point à verrouiller sera la définition exacte du cycle de promotion d'un projet expérimental vers une base de connaissances validée. developpe et inspire toi de cette discussion riche en sens et reflection sur ingestion et meto cognition

Tu veux transformer cette réflexion en une **doctrine d'architecture complète** : non seulement expliquer comment ingérer un document, mais concevoir un système capable de **savoir ce qu'il traite, pourquoi il le traite, comment il l'a transformé, ce qu'il peut affirmer et ce dont il doit encore douter.**

Architecture d'ingestion cognitive et métacognitive

De l'OCR documentaire à une gouvernance vérifiable de la connaissance

1. Changement de paradigme

Un système OCR-RAG classique suit généralement une chaîne relativement simple :

document → **extraction** → **découpage** → **embeddings** → **base vectorielle** → **recherche** → **réponse**

Cette chaîne fonctionne techniquement, mais elle ne garantit pas que les connaissances produites soient :

- exactes ;
- suffisamment contextualisées ;
- correctement segmentées ;
- attribuées à la bonne source ;
- toujours valides ;
- interprétées selon le bon angle ;
- récupérables dans les situations où elles sont réellement utiles.

Le système envisagé ici dépasse donc le cadre d'un simple pipeline documentaire.

Il devient une **infrastructure de transformation, d'expérimentation, de validation et de gouvernance de la connaissance**.

Son objectif n'est plus seulement de stocker du texte pour pouvoir le retrouver.

Son objectif est de construire une connaissance :

- isolée ;
- contextualisée ;
- traçable ;

- testable ;
- versionnée ;
- contestable ;
- réversible ;
- améliorable ;
- exploitable par différents modèles et différents modes de raisonnement.

Le principe fondamental peut être formulé ainsi :

Une information extraite n'est pas encore une connaissance fiable.

L'OCR produit du texte.

Le chunking produit des fragments.

L'embedding produit une représentation mathématique.

Le RAG produit des éléments potentiellement pertinents.

Mais seule une chaîne de validation, d'analyse et de contrôle permet de transformer ces éléments en connaissances utilisables avec confiance.

2. L'ingestion n'est pas une opération technique unique

L'erreur classique consiste à considérer l'ingestion comme un bouton :

Importer le document.

Dans cette architecture, l'ingestion est un **cycle de vie complet**.

Elle comprend au minimum les étapes suivantes :

1. identifier la source ;
2. préserver le document original ;
3. déterminer sa nature ;
4. choisir une stratégie d'extraction ;

5. mesurer la qualité de l'extraction ;
6. normaliser le contenu ;
7. reconstruire sa structure ;
8. segmenter le contenu ;
9. enrichir les segments ;
10. créer plusieurs représentations ;
11. indexer les connaissances ;
12. interroger la base produite ;
13. tester la récupération ;
14. détecter les erreurs ;
15. corriger ou réingérer ;
16. valider le projet ;
17. éventuellement promouvoir les connaissances.

L'ingestion devient donc une sorte de **procédure scientifique**.

Elle formule implicitement une hypothèse :

Avec cette méthode d'extraction, cette taille de chunks, ces métadonnées, cet embedding et cette méthode de recherche, je devrais pouvoir retrouver fidèlement les connaissances du document.

Cette hypothèse doit ensuite être testée.

3. Le projet comme unité fondamentale d'isolation

L'unité principale du système ne doit pas être le document, ni la base vectorielle, ni le modèle.

L'unité principale doit être le **projet d'ingestion**.

Un projet représente une expérimentation complète et autonome.

Il possède :

- une identité ;
- un objectif ;
- un corpus ;
- une configuration ;

- un pipeline ;
- des versions ;
- une base dédiée ;
- des index ;
- des journaux ;
- des tests ;
- des résultats ;
- des métriques ;
- un état de validation.

Le principe est :

Un projet correspond à un environnement de connaissance isolé.

Deux projets peuvent contenir le même document tout en appliquant des méthodes différentes.

Par exemple :

- le projet A utilise des chunks de 300 tokens ;
- le projet B utilise des chunks de 800 tokens ;
- le projet C utilise un découpage sémantique ;
- le projet D ajoute une analyse maïeutique ;
- le projet E génère des relations entre concepts ;
- le projet F utilise un autre modèle d'embedding.

Les résultats peuvent alors être comparés objectivement.

L'isolation empêche :

- la contamination entre corpus ;
- la confusion entre versions ;
- la perte de provenance ;
- le mélange de connaissances validées et expérimentales ;
- l'impossibilité de reproduire un résultat ;
- la destruction d'une base globale à la suite d'un mauvais paramétrage.

4. Les différents niveaux d'isolation

L'isolation ne doit pas uniquement concerner la base vectorielle.

Elle doit exister à plusieurs niveaux.

4.1 Isolation logique

Chaque donnée contient obligatoirement :

- un `project_id` ;
- un `document_id` ;
- une version ;
- un statut ;
- une provenance ;
- un identifiant de traitement.

Aucun objet ne doit pouvoir exister sans contexte de projet.

4.2 Isolation du stockage

Chaque projet peut disposer de :

- sa collection vectorielle ;
- son index lexical ;
- son stockage de fichiers ;
- ses tables de métadonnées ;
- ses caches ;
- ses résultats d'évaluation.

4.3 Isolation des configurations

Un projet conserve sa configuration exacte :

- moteur OCR ;
- règles de nettoyage ;
- stratégie de chunking ;
- taille cible ;
- overlap ;
- modèle d'embedding ;
- prompt d'enrichissement ;
- méthodes de raisonnement ;
- règles de reranking ;
- seuils de validation.

4.4 Isolation des permissions

Les opérations possibles peuvent varier selon le projet :

- lecture uniquement ;
- expérimentation ;
- modification ;
- suppression ;
- publication ;
- accès à des documents sensibles ;
- appels à des services externes.

4.5 Isolation temporelle

Chaque version du projet doit pouvoir être reconstruite.

Il faut pouvoir répondre à cette question :

Avec quelle version du corpus, du pipeline, du modèle et des paramètres cette réponse a-t-elle été produite ?

5. Les trois plans de l'architecture

L'architecture peut être divisée en trois plans distincts.

5.1 Le plan des données

Il contient ce qui est traité ou produit :

- documents ;
- pages ;
- blocs ;
- tableaux ;
- images ;
- chunks ;
- embeddings ;
- concepts ;
- relations ;
- questions-réponses ;
- synthèses ;
- sources ;
- citations ;
- scores ;

- résultats de tests.

5.2 Le plan de contrôle

Il décide de ce qui doit être exécuté :

- orchestration ;
- choix des modules ;
- gestion des dépendances ;
- gestion des états ;
- reprise après erreur ;
- politiques de sécurité ;
- contrôle des coûts ;
- validation des actions critiques.

Le Brain, le Registry, le State Manager et le Policy Checker appartiennent principalement à ce plan.

5.3 Le plan épistémique

Il gère la qualité et le statut des connaissances.

Il cherche à déterminer :

- ce que le système sait ;
- d'où cette connaissance provient ;
- dans quel contexte elle est vraie ;
- quel est son niveau de confiance ;
- quelles contradictions existent ;
- ce qui reste inconnu ;
- quelles vérifications sont nécessaires ;
- si la réponse peut être publiée.

Ce troisième plan constitue le cœur de la métacognition.

6. La métacognition appliquée à l'ingestion

La métacognition désigne ici la capacité du système à examiner son propre fonctionnement.

Il ne s'agit pas de prétendre que la machine possède une conscience.

Il s'agit de lui donner des mécanismes explicites lui permettant de répondre à des questions comme :

- Ai-je correctement compris le document ?
- Mon extraction est-elle complète ?
- Ai-je perdu une négation ?
- Ce tableau a-t-il été correctement reconstruit ?
- Ce chunk est-il compréhensible sans son contexte ?
- Mes métadonnées sont-elles suffisantes ?
- La source retrouvée répond-elle réellement à la question ?
- Est-ce une connaissance explicite ou une inférence ?
- Deux sources se contredisent-elles ?
- Mon niveau de confiance est-il justifié ?
- Dois-je répondre, rechercher davantage ou déclarer mon incertitude ?

La métacognition n'est donc pas un prompt ajouté à la fin.

Elle doit être distribuée dans toute l'architecture.

7. La boucle métacognitive

Chaque opération importante peut être entourée d'une boucle de contrôle.

Étape 1 — Prévision

Avant l'exécution, le système formule ses attentes.

Exemple :

- le document devrait contenir environ 240 pages ;
- la langue principale devrait être le français ;
- des tableaux sont probablement présents ;
- une stratégie OCR spécialisée sera nécessaire ;
- le coût prévu est estimé à 0,80 euro ;
- environ 1 500 chunks devraient être produits.

Étape 2 — Exécution

Le module réalise son traitement.

Étape 3 — Observation

Le système collecte les métriques réelles :

- pages extraites ;
- caractères détectés ;
- taux de confiance OCR ;
- anomalies ;
- durée ;
- consommation ;
- erreurs ;
- blocs non reconnus.

Étape 4 — Comparaison

Les résultats sont comparés aux attentes.

Exemple :

- 240 pages attendues, 237 extraites ;
- 12 tableaux détectés, seulement 3 reconstruits ;
- confiance OCR insuffisante sur 18 pages ;
- nombre de chunks anormalement faible.

Étape 5 — Décision

Le système choisit l'action appropriée :

- accepter ;
- avertir ;
- retraiter ;
- changer de module ;
- demander une validation humaine ;
- bloquer la promotion ;
- marquer une connaissance comme incertaine.

Étape 6 — Mémorisation

Le résultat est enregistré afin d'améliorer les prochaines décisions.

La boucle devient donc :

prévoir → exécuter → observer → comparer → décider → mémoriser

8. Les niveaux de connaissance

Toutes les informations ne doivent pas être stockées avec le même statut.

Une taxonomie épistémique permet de distinguer les éléments suivants.

8.1 Donnée brute

Exemple :

- image de page ;
- texte OCR non corrigé ;
- extraction HTML ;
- contenu du PDF.

Elle est conservée comme preuve d'origine.

8.2 Donnée normalisée

Le contenu a été nettoyé, mais son sens n'a pas encore été interprété.

8.3 Fait explicite

Le fait apparaît directement dans la source.

Exemple :

La durée du contrat est de douze mois.

8.4 Fait contextualisé

Le fait est enrichi de ses conditions :

- contrat concerné ;
- date ;
- juridiction ;
- version ;

- exceptions ;
- section du document.

8.5 Inférence

Le système déduit une conséquence qui n'est pas écrite explicitement.

Une inférence ne doit jamais être présentée comme une citation directe.

8.6 Hypothèse

Une possibilité est formulée, mais elle n'est pas encore vérifiée.

8.7 Contradiction

Deux connaissances incompatibles sont présentes.

Le système ne doit pas choisir silencieusement l'une d'elles.

8.8 Connaissance validée

Le fait a passé les critères de validation définis pour le projet.

8.9 Connaissance obsolète

La connaissance était valide, mais une version plus récente existe ou sa période de validité est dépassée.

9. Le chunk comme unité cognitive, pas seulement technique

Un chunk ne doit pas être considéré comme une simple chaîne de caractères limitée à un nombre de tokens.

Il doit être traité comme une **unité de connaissance potentielle**.

Un bon chunk doit :

- conserver un sens suffisamment autonome ;
- rester relié au contexte dont il dépend ;
- contenir sa provenance ;
- préserver les exceptions et négations ;

- identifier les concepts principaux ;
- déclarer ses dépendances ;
- pouvoir être retrouvé par plusieurs formulations de question.

Un chunk enrichi peut contenir :

```
{
  "chunk_id": "chk_01842",
  "project_id": "prd_ocr_001",
  "document_id": "doc_004",
  "document_version": "2.1",
  "page_start": 41,
  "page_end": 42,
  "section_path": [
    "Chapitre 3",
    "Conditions d'application",
    "Exceptions"
  ],
  "content": "Texte normalisé du passage...",
  "summary": "Résumé strictement fidèle au passage.",
  "explicit_facts": [],
  "inferences": [],
  "concepts": [],
  "entities": [],
  "keywords": [],
  "questions_answered": [],
  "prerequisites": [],
  "related_chunks": [],
  "exceptions": [],
  "temporal_scope": {},
  "jurisdiction": null,
  "source_quality": 0.96,
  "ocr_confidence": 0.91,
  "semantic_completeness": 0.88,
  "validation_status": "pending",
  "trace_id": "trc_a28f"
}
```

10. Le problème de la découpe

La découpe arbitraire peut détruire le sens.

Quelques risques :

- séparer une règle de son exception ;
- séparer une question de sa réponse ;
- découper un tableau ;
- perdre le titre qui contextualise un passage ;
- isoler un pronom de son référent ;
- séparer une définition du terme défini ;
- supprimer une négation en bordure de chunk ;
- fusionner deux sujets différents.

Le chunking doit donc combiner plusieurs stratégies :

1. découpe structurelle ;
2. découpe sémantique ;
3. limites de taille ;
4. chevauchement contrôlé ;
5. reconstruction du contexte ;
6. vérification de cohérence.

Une étape de métacognition peut évaluer chaque chunk :

- Est-il autonome ?
- Contient-il une idée principale identifiable ?
- Manque-t-il un élément présent dans le chunk précédent ?
- Contient-il plusieurs sujets incompatibles ?
- Une exception a-t-elle été séparée de sa règle ?
- Le titre de section doit-il être injecté dans ses métadonnées ?

11. Enrichissement maïeutique

L'enrichissement maïeutique ne doit pas servir à inventer des informations.

Il sert à révéler la structure intellectuelle du contenu.

Pour chaque passage, le système peut poser des questions comme :

- Quel problème ce passage cherche-t-il à résoudre ?
- Quelle affirmation principale contient-il ?
- Sur quelles conditions repose-t-elle ?

- Quelles exceptions sont mentionnées ?
- Quels termes nécessitent une définition ?
- Quelles conséquences sont explicites ?
- Quelles conséquences peuvent être raisonnablement inférées ?
- Quelles questions ce passage permet-il de résoudre ?
- Quelles informations manquent pour l'appliquer correctement ?
- Dans quel contexte cette information pourrait-elle être mal interprétée ?

Cette méthode transforme le chunk en une unité interrogable.

Elle améliore notamment :

- la génération de questions-réponses ;
- le retrieval ;
- la détection des ambiguïtés ;
- la recherche par intention ;
- la compréhension des conditions d'application.

12. Analyse critique et « gant de Kant »

L'analyse critique peut être structurée autour de plusieurs axes.

Validité

L'affirmation est-elle correctement soutenue par la source ?

Portée

Dans quelles conditions est-elle vraie ?

Limites

Quelles situations ne sont pas couvertes ?

Universalisation

La règle peut-elle être appliquée de manière générale ou dépend-elle fortement du contexte ?

Contradiction

Le passage est-il compatible avec d'autres parties du document ?

Responsabilité

Quelles conséquences produit l'utilisation de cette connaissance ?

Ambiguïté

Plusieurs interprétations sont-elles possibles ?

Cette couche critique ne remplace pas le contenu original.

Elle produit une représentation secondaire, clairement identifiée comme analyse.

13. Séparer les faits des interprétations

Une règle essentielle doit être imposée :

Le système ne doit jamais fusionner silencieusement la source et son interprétation.

Chaque information doit indiquer sa nature :

```
{
  "statement": "La règle semble exclure les contrats tempora
  "epistemic_type": "inference",
  "supported_by": [
    "chunk_21",
    "chunk_22"
  ],
  "confidence": 0.74,
  "validation_status": "unverified"
}
```

Cette séparation évite qu'une analyse générée par un modèle devienne progressivement une pseudo-vérité stockée dans le RAG.

14. Le Registry

Le Registry est le catalogue opérationnel du système.

Il décrit les modules disponibles, sans décider à leur place.

Pour chaque module, il peut enregistrer :

- identifiant ;
- nom ;
- version ;
- rôle ;
- capacités ;
- type d'entrée ;
- type de sortie ;
- contraintes ;
- dépendances ;
- coût ;
- latence ;
- fiabilité observée ;
- modèles compatibles ;
- données autorisées ;
- conditions d'activation ;
- politiques applicables ;
- état de santé.

Exemple :

```
{  
  "module_id": "semantic_chunker_v2",  
  "version": "2.3.1",  
  "capabilities": [  
    "semantic_segmentation",  
    "heading_preservation",  
    "exception_linking"  
  ],  
  "accepts": "normalized_document",  
  "returns": "chunk_collection",  
  "cost_profile": "low",  
  "risk_level": "low",  
  "status": "available"  
}
```

Le Registry répond à la question :

Quelles capacités sont disponibles ?

15. Le Brain

Le Brain répond à une autre question :

Quelles capacités faut-il utiliser maintenant ?

Il ne doit ni stocker les documents, ni effectuer directement tous les traitements.

Ses fonctions sont :

- comprendre l'objectif ;
- analyser le contexte ;
- examiner l'état du projet ;
- identifier les informations manquantes ;
- sélectionner les modules ;
- construire un plan ;
- prévoir les coûts ;
- soumettre les actions critiques au Policy Checker ;
- surveiller les résultats ;
- modifier le plan si nécessaire.

Le Brain doit être léger.

Il orchestre.

Il ne devient pas une « super-classe » monolithique possédant toute la logique du système.

16. Le Policy Checker

Le Policy Checker examine les actions avant leur exécution.

Il peut appliquer plusieurs catégories de règles.

Règles de sécurité

- interdiction de supprimer un projet validé sans confirmation ;
- interdiction d'envoyer un document confidentiel vers une API externe ;
- interdiction d'exécuter une commande hors du workspace ;

- interdiction d'écraser une version approuvée.

Règles financières

- plafond par opération ;
- plafond journalier ;
- modèle coûteux seulement après échec des modèles économiques ;
- estimation obligatoire avant traitement massif.

Règles épistémiques

- interdiction de promouvoir des chunks non sourcés ;
- citation obligatoire pour une réponse factuelle ;
- blocage si le taux d'OCR est insuffisant ;
- séparation obligatoire entre faits et inférences.

Règles opérationnelles

- tests obligatoires avant promotion ;
- sauvegarde avant migration ;
- journalisation avant action irréversible ;
- validation humaine pour les corpus sensibles.

17. Le State Manager

Le State Manager doit connaître l'état précis de chaque objet.

Par exemple :

```
document importé
→ OCR requis
→ OCR terminé
→ normalisation terminée
→ structure validée
→ chunking terminé
→ enrichissement partiel
→ embeddings terminés
→ index construit
→ tests en attente
→ validation refusée
```

Il évite les recalculs inutiles grâce à :

- des empreintes de contenu ;
- des versions ;
- des dépendances ;
- un graphe d'impact ;
- des caches ;
- des règles d'invalidation.

Lorsqu'une page est modifiée, le système détermine automatiquement les objets affectés.

Il peut recalculer :

- les blocs de la page ;
- les chunks concernés ;
- leurs embeddings ;
- les relations impactées ;
- certaines synthèses ;
- les tests associés.

Il ne relance pas aveuglément tout le projet.

18. Le graphe de provenance

Chaque connaissance doit posséder une chaîne de provenance complète.

Exemple :

Réponse finale

↓

Passage cité

↓

Chunk enrichi

↓

Bloc normalisé

↓

Résultat OCR

↓

Image de la page 42

↓

Document original version 1.3



Source d'acquisition

Cette traçabilité permet :

- l'audit ;
- la reproduction ;
- la correction ;
- la preuve ;
- l'identification des erreurs ;
- la comparaison entre versions.

Une réponse ne doit pas uniquement fournir une citation.

Elle doit pouvoir expliquer son **lignage documentaire**.

19. Le moteur d'évaluation

Une ingestion doit être évaluée par une batterie de tests.

19.1 Tests techniques

- toutes les pages ont-elles été traitées ?
- les caractères sont-ils lisibles ?
- les tableaux sont-ils intacts ?
- les images importantes sont-elles reliées au texte ?
- les métadonnées sont-elles complètes ?
- les embeddings ont-ils été créés ?

19.2 Tests de récupération

Le système génère ou utilise un jeu de questions de référence.

Il vérifie :

- si le bon chunk est retrouvé ;
- à quelle position ;
- avec quel score ;
- si plusieurs chunks nécessaires sont récupérés ;
- si une information concurrente perturbe la recherche.

Métriques possibles :

- Precision@K ;
- Recall@K ;
- Mean Reciprocal Rank ;
- taux de citation correcte ;
- taux de réponse non supportée ;
- taux de refus justifié.

19.3 Tests sémantiques

- le passage récupéré répond-il réellement à la question ?
- la réponse conserve-t-elle les exceptions ?
- la réponse distingue-t-elle faits et inférences ?
- les contradictions sont-elles signalées ?
- les limites du document sont-elles respectées ?

19.4 Tests adversariaux

Il faut également tester :

- les questions ambiguës ;
- les formulations inversées ;
- les fausses prémisses ;
- les questions hors corpus ;
- les demandes de certitude absolue ;
- les références à des versions obsolètes ;
- les questions qui mélangent plusieurs documents.

20. La capacité à ne pas répondre

La qualité d'un système fiable ne se mesure pas uniquement à sa capacité à produire une réponse.

Elle se mesure également à sa capacité à déclarer :

- information absente ;
- source insuffisante ;
- contradiction non résolue ;
- document obsolète ;
- extraction douteuse ;
- question trop ambiguë ;
- vérification externe nécessaire.

Un bon système ne dit pas seulement :

Voici la réponse.

Il peut dire :

Je ne dispose pas d'une preuve suffisante dans le projet
actuellement validé.

Cette capacité constitue une fonction essentielle de métacognition.

21. Les états du projet

Le cycle de vie d'un projet peut être formalisé ainsi :

DRAFT
CONFIGURED
INGESTING
EXTRACTED
NORMALIZED
CHUNKED
ENRICHED
INDEXED
TESTING
REVIEW_REQUIRED
VALIDATED
REJECTED
PROMOTION_PENDING
PROMOTED
DEPRECATED
ARCHIVED

Chaque transition doit posséder :

- des conditions ;
- des vérifications ;
- des permissions ;
- des événements ;
- des journaux ;
- une procédure de retour arrière.

Exemple :

La transition `TESTING → VALIDATED` pourrait exiger :

- couverture documentaire supérieure à 99 % ;
 - confiance OCR moyenne supérieure au seuil ;
 - absence d'erreur critique ;
 - réussite du jeu de tests ;
 - citations vérifiées ;
 - validation humaine facultative ou obligatoire selon le domaine.
-

22. La promotion vers une base certifiée

La promotion ne consiste pas simplement à copier tous les embeddings vers une collection globale.

Elle doit être une procédure contrôlée.

Conditions de promotion

- projet validé ;
- version figée ;
- sources accessibles ;
- provenance complète ;
- schémas conformes ;
- résultats de tests archivés ;
- règles de licence respectées ;
- absence de conflit non résolu ;
- politique de confidentialité respectée.

Contenu promu

Le système peut promouvoir :

- le corpus complet ;
- certains documents ;
- certains chunks ;
- uniquement les faits validés ;
- les relations entre concepts ;
- les jeux de questions-réponses ;
- les méthodes d'ingestion performantes.

Promotion immuable

Les objets validés ne doivent pas être modifiés silencieusement.

Une correction produit une nouvelle version.

La version antérieure reste consultable et peut être dépréciée.

23. La base globale n'est pas une poubelle centrale

Une base globale mal gouvernée devient rapidement inutilisable.

Elle finit par contenir :

- plusieurs versions contradictoires ;
- des connaissances non vérifiées ;
- des doublons ;
- des inférences présentées comme des faits ;
- des documents obsolètes ;
- des erreurs OCR ;
- des fragments sans provenance.

La base commune doit plutôt être conçue comme un **registre de connaissances certifiées**.

Elle ne reçoit pas tout.

Elle reçoit uniquement ce qui a franchi les étapes prévues.

24. L'optimiseur de fond

L'optimiseur analyse progressivement la qualité de la base.

Il peut :

- détecter les doublons ;
- identifier les chunks orphelins ;
- repérer les relations manquantes ;
- calculer la dérive entre versions ;
- signaler les données obsolètes ;
- proposer un rechunking ;
- détecter une baisse de performance ;

- recommander un nouvel embedding ;
- reconstruire un index ;
- proposer une fusion de concepts.

Mais il ne doit pas modifier directement une connaissance certifiée.

Il produit une proposition :

```
{  
  "proposal_type": "merge_duplicate_chunks",  
  "affected_objects": [  
    "chk_124",  
    "chk_891"  
  ],  
  "estimated_benefit": 0.18,  
  "risk_level": "medium",  
  "requires_validation": true  
}
```

25. Une architecture anti-fragile

Une architecture robuste résiste aux erreurs.

Une architecture anti-fragile utilise les erreurs pour s'améliorer.

Chaque échec peut alimenter un registre d'apprentissage :

- type d'erreur ;
- module concerné ;
- configuration ;
- contexte ;
- correction appliquée ;
- résultat de la correction ;
- règle préventive produite.

Exemple :

Erreur :

les tableaux sur deux pages sont mal reconstruits.

Correction :

activation d'un moteur de détection de structure spécialisé.

Apprentissage :

pour les documents comportant plus de 10 % de pages tabulaires
utiliser automatiquement la stratégie `table_aware_ocr`.

Ainsi, le système ne mémorise pas uniquement des connaissances documentaires.

Il mémorise aussi des **connaissances sur sa propre manière de travailler**.

26. Deux mémoires différentes

L'architecture doit distinguer deux mémoires.

Mémoire de contenu

Elle contient :

- documents ;
- faits ;
- chunks ;
- concepts ;
- relations ;
- réponses ;
- citations.

Mémoire procédurale

Elle contient :

- stratégies efficaces ;
- configurations performantes ;
- erreurs rencontrées ;
- règles d'activation ;
- coûts observés ;
- performances par type de document ;
- préférences validées ;
- séquences de modules fiables.

Cette seconde mémoire permet au Brain de choisir progressivement de meilleures stratégies.

27. Le journal de décision

Le système ne doit pas seulement enregistrer ce qu'il a fait.

Il doit enregistrer pourquoi il l'a fait.

Exemple :

```
{
  "decision_id": "dec_00812",
  "project_id": "legal_2026_04",
  "objective": "extract_structured_knowledge",
  "observations": [
    "document_scanned",
    "contains_complex_tables",
    "language_french"
  ],
  "considered_options": [
    "basic_ocr",
    "layout_aware_ocr",
    "vision_model"
  ],
  "selected_option": "layout_aware_ocr",
  "reasons": [
    "better_table_preservation",
    "acceptable_cost",
    "validated_on_similar_documents"
  ],
  "predicted_cost": 0.42,
  "predicted_duration_seconds": 310,
  "policy_status": "approved",
  "trace_id": "trc_338f"
}
```

Ce journal transforme une suite opaque d'appels en processus auditable.

28. Architecture modulaire proposée

CONTROL TOWER

|

├─ Project Manager

| └─ création du workspace

- | | | cycle de vie
- | | | versions
- | | | permissions
- |
- | | Brain / Planner
 - | | | analyse de l'objectif
 - | | | sélection des capacités
 - | | | génération du plan
 - | | | adaptation du plan
- |
- | | Registry
 - | | | modules
 - | | | contrats
 - | | | versions
 - | | | coûts
 - | | | performances
- |
- | | Policy Checker
 - | | | sécurité
 - | | | confidentialité
 - | | | coûts
 - | | | règles épistémiques
- |
- | | State Manager
 - | | | états
 - | | | dépendances
 - | | | cache
 - | | | invalidation
- |
- | | Ingestion Engine
 - | | | acquisition
 - | | | OCR
 - | | | parsing
 - | | | normalisation
 - | | | reconstruction
- |
- | | Knowledge Processing
 - | | | chunking
 - | | | enrichissement
 - | | | maïeutique
 - | | | analyse critique
 - | | | extraction de faits
 - | | | génération Q/R

- |
- | — Indexing Layer
 - | | — embeddings
 - | | — index lexical
 - | | — index vectoriel
 - | | — graphe
 - | | — index temporel
- |
- | — Retrieval Engine
 - | | — compréhension de requête
 - | | — recherche hybride
 - | | — filtrage
 - | | — reranking
 - | | — reconstruction du contexte
- |
- | — Epistemic Validator
 - | | — qualité des preuves
 - | | — contradictions
 - | | — niveau de confiance
 - | | — faits / inférences
 - | | — décision de réponse
- |
- | — Evaluation Arena
 - | | — jeux de tests
 - | | — comparaison de pipelines
 - | | — benchmarks
 - | | — tests adversariaux
 - | | — classement des versions
- |
- | — Background Optimizer
 - | | — détection des doublons
 - | | — maintenance des index
 - | | — dérive
 - | | — obsolescence
 - | | — propositions d'amélioration

29. Le contrat universel des modules

Tous les modules devraient respecter une enveloppe commune.

```
{
  "request": {
```

```

    "project_id": "project_001",
    "operation_id": "op_9842",
    "module_id": "ocr_layout",
    "module_version": "1.3.0",
    "input_refs": [],
    "parameters": {},
    "constraints": {
        "max_cost": 1.0,
        "external_processing_allowed": false
    },
    "trace_id": "trace_883"
},
"response": {
    "status": "success",
    "output_refs": [],
    "observations": [],
    "warnings": [],
    "errors": [],
    "metrics": {
        "duration_ms": 0,
        "estimated_cost": 0,
        "actual_cost": 0,
        "confidence": 0
    },
    "epistemic_status": {
        "quality": "acceptable",
        "requires_review": false
    }
}
}

```

Cette enveloppe commune rend possible :

- l'interchangeabilité ;
- les tests unitaires ;
- le remplacement d'un fournisseur ;
- l'enregistrement des performances ;
- la reprise après erreur ;
- la comparaison entre versions.

30. Exemple de traitement complet

Demande

Ingérer un manuel technique de 400 pages.

Analyse initiale du Brain

- PDF mixte : texte et scans ;
- nombreuses illustrations ;
- tableaux de maintenance ;
- procédures contenant des avertissements ;
- nécessité de préserver les séquences d'étapes ;
- confidentialité moyenne ;
- budget limité.

Plan proposé

1. conserver le PDF original ;
2. détecter les pages texte et image ;
3. appliquer l'OCR uniquement aux pages scannées ;
4. reconstruire les blocs ;
5. identifier les procédures ;
6. préserver les encadrés de sécurité ;
7. découper par procédure plutôt que par taille fixe ;
8. produire des chunks parents et enfants ;
9. extraire équipements, pannes, symptômes et actions ;
10. générer un index vectoriel et lexical ;
11. créer des questions de test ;
12. mesurer la récupération ;
13. vérifier les citations ;
14. soumettre le projet à validation.

Contrôle métacognitif

Le système constate que :

- 28 pages ont une confiance OCR faible ;
- 7 tableaux sont incomplets ;
- 14 procédures ont été divisées ;
- plusieurs avertissements ont été séparés de l'action correspondante.

Il ne valide pas l'ingestion.

Il déclenche :

- un OCR alternatif sur les pages douteuses ;
- une reconstruction spécialisée des tableaux ;
- un regroupement des procédures ;
- une règle empêchant la séparation des avertissements.

Le projet est ensuite testé à nouveau.

Cette boucle illustre la différence entre :

Le pipeline s'est exécuté.

et :

La connaissance produite est suffisamment fiable pour être utilisée.

31. Le moteur de questions-réponses comme instrument de validation

La génération de questions-réponses ne sert pas uniquement à enrichir la base.

Elle peut servir à tester l'ingestion.

Pour chaque section, le système produit plusieurs catégories de questions :

Questions factuelles

- Quelle est la tension nominale de l'équipement ?
- Quel est le délai prévu ?
- Qui doit approuver l'opération ?

Questions conditionnelles

- Dans quelles conditions cette procédure s'applique-t-elle ?
- Quand cette règle ne doit-elle pas être utilisée ?

Questions négatives

- Qu'est-ce qui est explicitement interdit ?
- Quelle opération ne doit jamais être effectuée ?

Questions relationnelles

- Quelle est la relation entre le symptôme A et la panne B ?
- Quelle étape dépend de la validation précédente ?

Questions adversariales

- Peut-on appliquer cette procédure dans tous les cas ?
- Le document affirme-t-il réellement cette conclusion ?

Chaque réponse attendue reste liée à un ou plusieurs passages sources.

Le système mesure ensuite si sa propre base est capable de retrouver les preuves nécessaires.

32. Les métriques essentielles

Il est impossible de piloter cette architecture sans métriques.

Qualité de l'extraction

- taux de pages extraites ;
- confiance OCR ;
- erreurs de caractères ;
- blocs perdus ;
- tableaux reconstruits ;
- structure préservée.

Qualité des chunks

- autonomie sémantique ;
- cohérence ;
- densité d'information ;
- taux de chunks orphelins ;
- taux de règles séparées de leurs exceptions.

Qualité du retrieval

- Recall@K ;

- Precision@K ;
- rang du passage pertinent ;
- taux de contexte suffisant ;
- taux de citations correctes.

Qualité épistémique

- taux d'affirmations supportées ;
- taux d'inférences correctement identifiées ;
- taux de contradictions détectées ;
- taux de refus appropriés ;
- calibrage de la confiance.

Performance opérationnelle

- durée ;
 - coût ;
 - consommation de tokens ;
 - utilisation CPU/GPU ;
 - taille des index ;
 - taux de réutilisation du cache.
-

33. Les principaux risques architecturaux

Risque 1 — Sur-enrichissement

Un enrichissement excessif peut :

- multiplier les coûts ;
- créer du bruit ;
- produire des interprétations fragiles ;
- dégrader le retrieval.

Chaque enrichissement doit démontrer une utilité mesurable.

Risque 2 — Brain monolithique

Si le Brain contient toute la logique, il devient impossible à tester et à faire évoluer.

Il doit rester un orchestrateur.

Risque 3 — Faux score de confiance

Un modèle peut produire un nombre sans que ce nombre corresponde à une probabilité réelle.

La confiance doit être fondée sur des critères observables :

- nombre de sources ;
- qualité des sources ;
- cohérence ;
- couverture ;
- performance passée ;
- présence de contradictions.

Risque 4 — Promotion trop rapide

Une ingestion techniquement réussie n'est pas nécessairement fiable.

Risque 5 — Mélange des couches

Les données sources, analyses, hypothèses et synthèses doivent rester distinguées.

Risque 6 — Complexité prématurée

Toutes les couches ne doivent pas être développées simultanément.

Le système doit progresser par niveaux de maturité.

34. Développement par niveaux de maturité

Niveau 1 — Ingestion fiable

Objectif :

- créer un projet ;
- ajouter un document ;
- extraire ;
- chunker ;
- indexer ;
- interroger ;
- citer ;

- journaliser.

Niveau 2 — Isolation et comparaison

Objectif :

- plusieurs projets ;
- plusieurs configurations ;
- métriques ;
- Arena comparative ;
- versionnement.

Niveau 3 — Gouvernance

Objectif :

- Policy Checker ;
- états de validation ;
- promotion ;
- permissions ;
- audit.

Niveau 4 — Métacognition

Objectif :

- prévisions ;
- auto-évaluation ;
- détection des anomalies ;
- faits et inférences ;
- contradictions ;
- refus justifiés.

Niveau 5 — Auto-amélioration contrôlée

Objectif :

- apprentissage procédural ;
- recommandations automatiques ;
- optimisation en arrière-plan ;
- adaptation aux types de documents ;
- propositions de nouveaux outils.

L'ordre est important.

Il ne faut pas tenter de construire directement le niveau 5 sans avoir stabilisé le niveau 1.

35. Formulation du produit

Le produit peut être défini ainsi :

Une plateforme d'ingestion cognitive permettant de transformer des corpus documentaires en bases de connaissances isolées, testées, traçables et gouvernées, avant leur éventuelle promotion vers un référentiel certifié.

Sa valeur ne repose pas seulement sur l'OCR.

Elle repose sur la capacité à garantir :

- la séparation des projets ;
 - la reproductibilité ;
 - l'évaluation comparative ;
 - la provenance ;
 - la maîtrise des coûts ;
 - la détection des erreurs ;
 - la promotion contrôlée ;
 - la transparence des décisions.
-

36. Vision finale

Cette architecture ne cherche pas à fabriquer un modèle qui prétend tout savoir.

Elle cherche à fabriquer un système qui sait organiser son ignorance.

Il sait :

- ce qu'il possède ;
- ce qu'il a transformé ;
- ce qu'il a vérifié ;
- ce qu'il suppose ;
- ce qui se contredit ;

- ce qui doit être recalculé ;
- ce qui peut être publié ;
- ce qui doit rester expérimental.

La véritable métacognition du système ne réside donc pas dans un discours simulant une conscience.

Elle réside dans la discipline de son architecture.

Une architecture métacognitive est une architecture qui peut examiner :

- ses entrées ;
- ses choix ;
- ses outils ;
- ses résultats ;
- ses limites ;
- ses erreurs ;
- ses conséquences.

Le système OCR–RAG devient alors une véritable tour de contrôle de la connaissance.

Il ne se contente plus d'absorber des documents.

Il organise un processus contrôlé dans lequel une donnée peut progressivement devenir :

extraite → structurée → contextualisée → testée → validée → gouvernée → exploitable

C'est ce passage progressif, documenté et réversible, qui transforme un pipeline technique en infrastructure cognitive fiable.

Évaluation

Cohérence de la vision : 4,9/5

Le concept est très solide parce qu'il réunit quatre dimensions souvent séparées : ingestion documentaire, expérimentation, gouvernance et métacognition. Le principal danger n'est pas une faiblesse conceptuelle, mais une **surconstruction prématurée**.

La prochaine action rationnelle est de figer un **MVP de niveau 1** : un projet isolé, un document, une chaîne d'ingestion, une base dédiée, dix questions de contrôle et un rapport de validation. Ensuite seulement, on ajoute le Brain, le Policy Checker et les boucles métacognitives.

Do you like this personality?